

Simulation Modeling and Analysis

Dr. Md. Nasim Adnan

INTRODUCTION

- In studying the simulation examples, the reader probably noticed several features needed in programming most discrete-event simulation models, including:
 - Generating random numbers, that is, observations from a $U(0,1)$ probability distribution
 - Generating random variates from a specified probability distribution (e.g., exponential)
 - Advancing simulated time
 - Determining the next event from the event list and passing control to the appropriate block of code
 - Adding records to, or deleting records from, a list
 - **Collecting output statistics and reporting the results**
 - Detecting error conditions

COMPARISON OF SIMULATION PACKAGES WITH PROGRAMMING LANGUAGES

- One of the most important decisions a modeler or analyst must make in performing a simulation study concerns the choice of software.
- If the selected software is not flexible enough or is too difficult to use, then the simulation project may produce erroneous results or may not even be completed.
- The following are some advantages of using a simulation package rather than a general-purpose programming language:
 - Simulation packages automatically provide most of the features needed to build a simulation model, resulting in a significant decrease in “programming” time and a reduction in overall project cost.

COMPARISON OF SIMULATION PACKAGES WITH PROGRAMMING LANGUAGES

- They provide a natural framework for simulation modeling. Their basic modelling constructs are more closely akin to simulation than are those in a general-purpose programming language like C.
- ***Simulation models are generally easier to modify and maintain when written in a simulation package.***
- They provide better error detection because many potential types of errors are checked for automatically. Since fewer modeling constructs need to be included in a model, the chance of making an error will probably be smaller.

COMPARISON OF SIMULATION PACKAGES WITH PROGRAMMING LANGUAGES

- On the other hand, some simulation models (particularly for defense-related applications) are still written in a general-purpose programming language.
- Some advantages of such a choice are as follows:
 - Most modelers already know a programming language, but this is often not the case with a simulation package.
 - A simulation model efficiently written in C, C++, Java or Python may require less execution time than a model developed in a simulation package.
 - This is so because a simulation package is designed to address a wide variety of systems with one set of modeling constructs, whereas a Python program can be more closely tailored to a particular application.

COMPARISON OF SIMULATION PACKAGES WITH PROGRAMMING LANGUAGES

- Programming languages may allow *greater programming flexibility than certain simulation packages*.
- The programming languages C++, Java and Python are object-oriented, which is of considerable importance to many analysts and programmers, such as those in the defense industry.
- On the other hand, most simulation packages are not truly object-oriented.
- Software cost is generally lower, but total project cost may not be.
- Although there are advantages to using both types of software, we believe, in general, that a modeler would be prudent to give serious consideration to using a simulation package.

CLASSIFICATION OF SIMULATION SOFTWARE

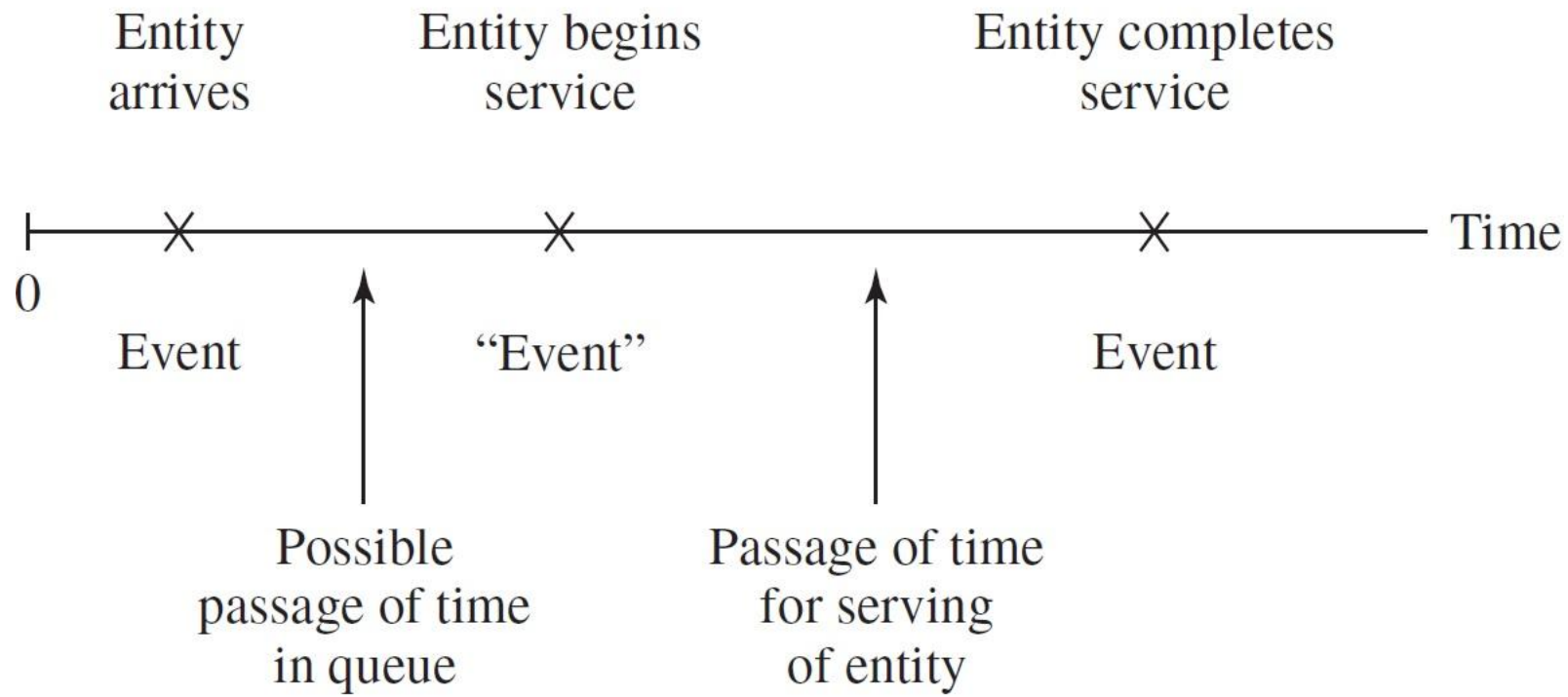
- General-Purpose vs. Application-Oriented Simulation Packages:
- There are two main types of simulation packages for ***discrete-event simulation***, namely, general-purpose simulation software and application-oriented simulation software
- A general-purpose simulation package can be used for any application, but might have special features for certain ones (e.g., for manufacturing or process reengineering).
- On the other hand, an application-oriented simulation package is designed to be used for a certain class of applications such as manufacturing, health care, or communications networks (Cisco Packet Tracer).

Modeling Approaches: event-scheduling approach

- So far we have used the **event-scheduling approach** to discrete-event simulation modeling.
- A system is modeled by identifying its characteristic events and then writing a set of event routines that give a detailed description of the state changes taking place at the time of each event.
- The simulation **evolves over time** by executing the events in increasing order of their time of occurrence.
- Here a basic property of an event routine is that **no simulated time passes during its execution**.

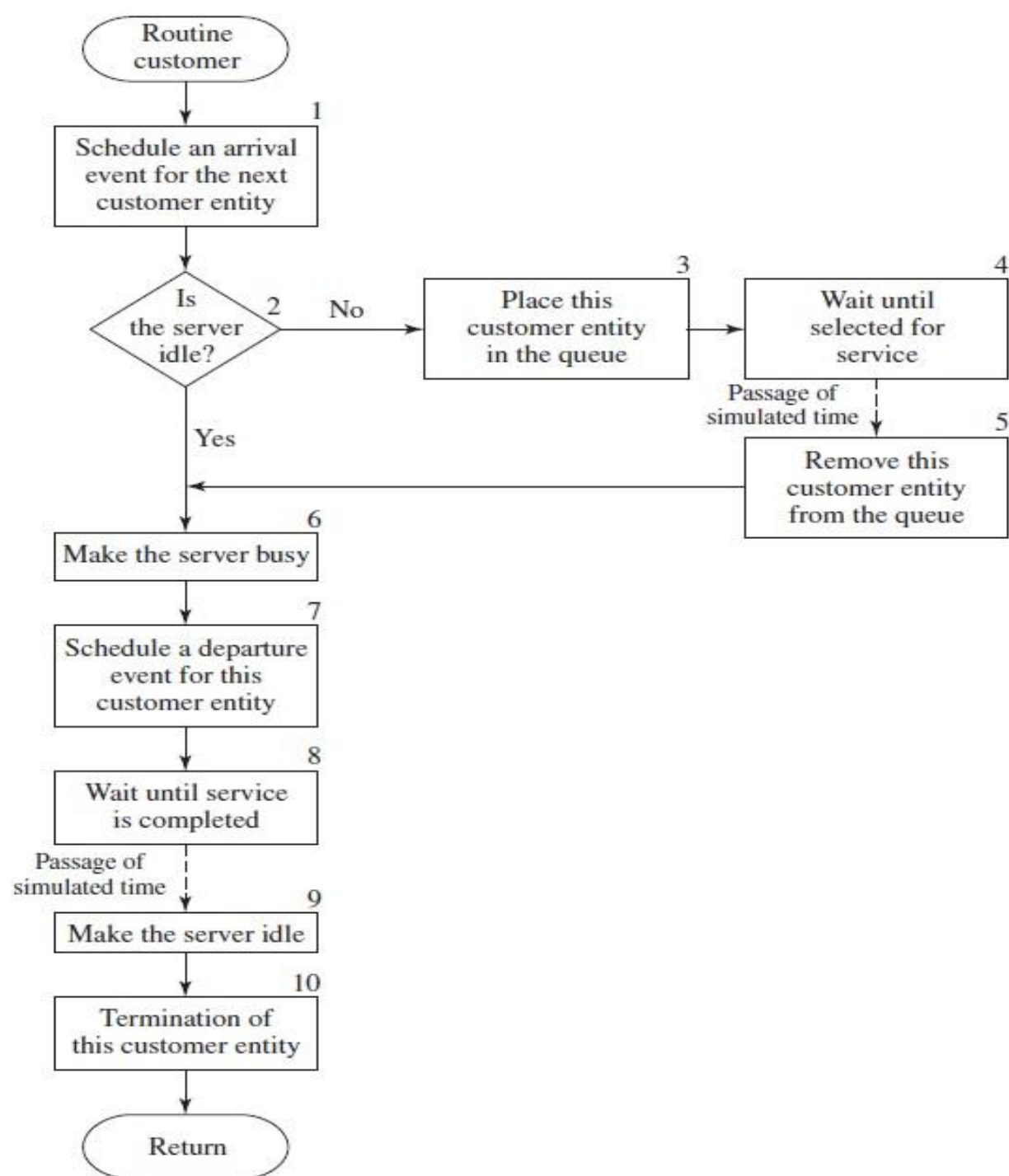
Modeling Approaches: process approach

- On the other hand, most contemporary simulation packages use the process approach to simulation modeling.
- A process is a **time-ordered sequence of interrelated events** separated by intervals of time, which describes **the entire experience of an “entity” as it flows through a “system.”**
- The process corresponding to an entity arriving to and being served at a single server is shown in the next figure.
- A system or simulation model may have several different types of processes. Corresponding to each process in the model, there is a process “routine” that describes the entire history of its “process-entity” as it moves through.



• • •

- To illustrate the nature of the process approach more clearly, the next figure gives a flowchart for a prototype customer-process routine in the case of a single server queueing system.
- Entry into this routine at block 1 corresponds to the arrival event for a customer entity that is the most imminent event in the event list.
- At block 1 an arrival event record is placed in the event list for the next customer entity to arrive. (This next customer entity will arrive at a time equal to the time the current customer entity arrives plus an interarrival time.)



• • •

- To determine whether the customer entity currently arriving can begin service, a check is made (at block 2) to see whether the server is idle.
- If the server is busy, this customer entity is placed at the end of the queue (block 3) and is made to wait (at block 4) until selected for service at some undetermined time in the future (This is called a conditional wait.)
- Control is then returned to the “timing routine” to determine which customer entity’s event is the most imminent now.

CLASSIFICATION OF SIMULATION SOFTWARE

- When this customer entity (the one made to wait at block 4) is activated at some point in the future (when it is first in queue and another customer completes service and makes the server idle), it is removed from the queue at block 5 and begins service immediately, thereby making the server busy (block 6).
- A customer entity arriving to find the server idle also begins service immediately (at block 6); in either case, we are now at block 7.
- There the departure time for the customer beginning service is determined, and a corresponding event record is placed in the event list.
- This customer entity is then made to wait (at block 8) until its service has been completed. This is an unconditional wait, since its activation time is known.

CLASSIFICATION OF SIMULATION SOFTWARE

- Control is returned to the timing routine to determine what customer entity will be processed next.
- When the customer made to wait at block 8 is activated at the end of its service, this makes the server idle at block 9 (allowing the first customer in the queue to become active immediately), and then this customer is removed from the system at block 10.

Common Modeling Elements

- Simulation packages typically include entities, attributes, resources, and queues as part of their modeling framework.
- An entity is created, travels through some part of the simulated system, and then is usually destroyed.
- Entities are distinguished from each other by their attributes, which are pieces of information stored with the entity. As an entity moves through the simulated system, it requests the use of resources. If a requested resource is not available, then the entity joins a queue.
- The entities in a particular queue may be served in a FIFO (first-in, first-out) manner, served in a LIFO (last-in, first-out) manner, or ranked on some attribute in increasing or decreasing order.

DESIRABLE SOFTWARE FEATURES

- There are numerous features to consider when selecting simulation software. We categorize these features as being in one of the following groups:
 - General capabilities (including modeling flexibility and ease of use)
 - Hardware and software requirements
 - Animation and dynamic graphics
 - **Statistical capabilities**
 - Customer support and documentation
 - Output reports and graphics
- We now discuss each group of features in turn.

General Capabilities

- In our opinion, the most important feature for a simulation-software product to have is modeling flexibility or, in other words, the ability to model a system whose operating procedures can have any amount of complexity.
- Note that no two systems are exactly alike. Thus, a simulation package that relies on a fixed number of modelling constructs with no capability to do some kind of programming in any manner is bound to be inadequate for certain systems encountered in practice.
- Ideally, it should be possible to model any system using only the constructs provided in the software—it should not be necessary to use routines written in a programming language such as C.

General Capabilities

- The following are some specific capabilities that make a simulation product flexible:
 - Ability to define and change attributes for entities and also global variables, and to use both in decision logic (e.g., if-then-else constructs)
 - Ability to use mathematical expressions and mathematical functions (logarithms, exponentiation, etc.)
 - Ability to create new modeling constructs and to modify existing ones, and to store them in libraries for use in current and future models

General Capabilities

- The second most important feature for a simulation product is ease of use (and ease of learning), and many contemporary simulation packages have a graphical user interface to facilitate this.
- The software product should have modelling constructs (e.g., icons or blocks) that are neither too “primitive” nor too “macro.”
- In the former case, a large number of constructs will be required to model even a relatively simple situation; in the latter case, each construct’s dialog box will contain an excessive number of options if it is to allow for adequate flexibility.

General Capabilities

- Hierarchical modeling is useful in modeling complex systems. Hierarchy allows a user to combine several basic modeling constructs into a new higher-level construct.
- The software should have good debugging aids such as an interactive debugger. A powerful debugger allows the user to do things such as:
 - Follow a single entity through the model to see if it is processed correctly
 - See the state of the model every time a particular event occurs
 - Set the value of certain attributes or variables to “force” an entity down a logical path that occurs with small probability

General Capabilities

- Fast model execution speed is important for certain applications such as large military models and models in which a large number of entities must be processed (e.g., for a high-speed communications network).
- It is desirable to be able to develop user-friendly model “front ends” when the simulation model is to be used by someone other than the model developer.
- This capability allows the developer to create an interface by which the nonexpert user can easily enter model parameters such as the mean service time or how long to run the simulation.

General Capabilities

- Most simulation software vendors offer a run-time version of their software, which, roughly speaking, allows the user to change model data but not logic by employing a user-friendly “front end.”
- Applications of a run-time version include:
 - Allowing a person in one division of an organization to run a model that was developed by a person in another division who owns a developmental version of the simulation software
 - Sales tool for equipment suppliers or system integrators
 - Training
- Note that a run-time license generally has a considerably lower cost than a normal developmental license or is free.

Hardware and Software Requirements

- In selecting simulation software, one must consider what computer platforms the software is available for. Almost all software is available for Windows-based PCs, and some products are also available for Apple computers.
- If a software package is available for several platforms, then it should be compatible across platforms.
- The amount of RAM required to run the software should be considered as well as what operating systems are supported.
- It is highly desirable if independent replications of a simulation model can be made simultaneously on multiple processor cores or on networked computers.

Animation and Dynamic Graphics

- The availability of built-in animation is one of the reasons for the increased use of simulation modeling.
- In an animation, key elements of the system are represented on the screen by icons that dynamically change position, color, and shape as the simulation model evolves through time.
- For example, in a manufacturing system, an icon representing a forklift truck will change position when there is a corresponding change in the model, and an icon representing a machine might change color when the machine changes state (e.g., idle to busy) in the model.

Animation and Dynamic Graphics

- The following are some of the uses of animation:
 - Communicating the essence of a simulation model (or simulation itself) to a manager or to other people who may not be aware of (or care about) the technical details of the model
 - Debugging the simulation computer program
 - Showing that a simulation model is not valid
 - Suggesting improved operational procedures for a system (some things may not be apparent from looking at just the simulation's numerical results)
 - Training operational personnel
 - Promoting communication among the project team

Statistical Capabilities

- If a simulation product does not have good statistical-analysis features, then it is impossible to obtain correct results from a simulation study. First, the software must have a good random-number generator, that is, a mechanism for generating independent observations from a uniform distribution on the interval $[0, 1]$.
- Note that not all random-number generators found on computers or in software products have acceptable statistical properties.

Customer Support and Documentation

- The simulation software vendor should provide public training on the software on a regular basis, and it should also be possible to have customized training presented at the client's site.
- Good technical support is extremely important for questions on how to use the software and in case a bug in the software is discovered.
- Technical support, which can be in the form of telephone help, should be such that a response is received in at most one day.

Customer Support and Documentation

- Good documentation is a crucial requirement for using any software product. It should be possible, in our opinion, to learn a simulation package without taking a formal training course.
- Generally, there will be a user's guide or reference manual.
- There should be numerous detailed examples available. Most products now have context-dependent online help, which we consider very important.
- Several products have a library of “mini examples” to illustrate the various modelling constructs.

Output Reports and Graphics

- Standard reports should be provided for the estimated performance measures.
- It should also be possible to customize reports, perhaps for management presentations.
- Since a simulation product should be flexible enough so that it can compute estimates of user-defined performance measures, it should also be possible to write these estimates into a custom report.
- For each performance measure (e.g., time in system for a factory), the average observed value, the minimum observed value, and the maximum observed value are usually given.
- If a standard deviation is also given (based on one simulation run), then the user should be sure that it is based on a statistically acceptable method, or else it should be viewed as highly suspect.

Output Reports and Graphics

- The simulation product should provide a variety of (static) graphics.
- First, it should be possible to make a histogram for a set of observed data.
- In a time plot one or more key system variables (e.g., the numbers in certain queues) are plotted over the length of the simulation, providing a long-term indication of the dynamic behavior of the simulated system.
- Some simulation products allow the simulation results to be presented in bar charts or pie charts.
- Finally, a correlation plot is a useful way to measure the dependence in the output data produced from one simulation run.